

CS182 Special Participation B: HW0

Alena Chao

December 5, 2025

Executive Summary

Across the responses, the model consistently produces very procedural, teacher-style outputs: it always gives the canonical textbook implementation, includes brief explanations of why each line is correct, and avoids offering alternative designs unless prompted. For multi-stage tasks (e.g., affine $\rightarrow$ ReLU $\rightarrow$ two-layer net), the model always follows a deterministic pattern: provide the code block, then bullet-point interpretations, then optional notes. When moving into higher-level tasks (e.g., training with a Solver), its behavior shifts slightly toward suggesting hyperparameters and giving “good practice” rather than just returning the minimal requested snippet. The most common type of comment was noting the shapes of outputs, suggesting emphasis on common mistakes or debugging points. Overall, the model tends to assume the user wants both the solution and the conceptual rationale, resulting in answers that are more verbose, more tutorial-like, and more explanatory than the user’s minimal requests. In general, the model did not struggle on the tasks as it was able to complete them in one shot. This is most likely due to the very structured and fundamental nature of the tasks.

6. Fully-Connected Neural Networks Assignment

ANNOTATION: From the first interaction I can see that the model likes to explain the code step by step, even if the docstring is pretty informative. It is quite particular about shapes.

User

Setup Environment & Affine Layer: Forward

[...Environment setup code omitted for brevity...]

Open the file `deeplearning/layers.py` and implement the `affine_forward` function.

```
def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.
    Inputs:
    x: Input data, of shape (N, d_1, ..., d_k)
    w: Weights, of shape (D, M)
    b: Biases, of shape (M,)
    """
    out = None
    # TODO: Implement the affine forward pass.
    cache = (x, w, b)
    return out, cache
```

ChatGPT

We can implement the `affine_forward` function by first reshaping the input x into a 2D matrix (with each example flattened into a row), performing the matrix multiplication with the weights

w , and then adding the bias b .
Here's a correct implementation:

```
def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.
    """
    # Reshape x into (N, D) where D = d_1 * ... * d_k
    N = x.shape[0]
    X_flat = x.reshape(N, -1) # flatten each input example

    # Compute output
    out = X_flat.dot(w) + b # shape (N, M)

    # Store cache for backward pass
    cache = (x, w, b)
    return out, cache
```

Key points:

- `x.reshape(N, -1)` flattens each example while keeping the batch size N .
- Matrix multiplication `X_flat.dot(w)` gives (N, M) .
- Adding b automatically broadcasts across the batch.

User

Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

ChatGPT

Here's how you can implement the `affine_backward` function. The key is to carefully apply the chain rule and remember to reshape dx back to the shape of the original input x .

```
def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.
    """
    x, w, b = cache
    N = x.shape[0]
    X_flat = x.reshape(N, -1) # flatten x to shape (N, D)

    # Gradients
    dw = X_flat.T.dot(dout) # (D, N) dot (N, M) -> (D, M)
    db = np.sum(dout, axis=0) # sum over batch -> (M,)
    # (N, M) dot (M, D) -> (N, D) -> reshape to x's shape
    dx = dout.dot(w.T).reshape(x.shape)

    return dx, dw, db
```

Notes:

- dw comes from $\frac{\partial L}{\partial w} = X^T \cdot dout$.
- db comes from summing $dout$ over the batch (`axis=0`).
- dx comes from $dout \cdot w^T$, then reshaped.

ANNOTATION: sometimes the model does not strictly follow the skeleton code, usually to make code more concise.

User

ReLU layer: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function.

ChatGPT

Here's a simple and correct implementation for the ReLU forward pass:

```
def relu_forward(x):  
    """  
    Computes the forward pass for a layer of rectified linear units (ReLU).  
    """  
    out = np.maximum(0, x) # element-wise ReLU  
    cache = x # store input for backward pass  
    return out, cache
```

Notes:

- ReLU just outputs the input if it's positive, and 0 otherwise.
- The cache is simply x because we need it to compute the gradient in the backward pass.

ANNOTATION: it seems like for more complex tasks, the model will also output explanations for its code rather than just the code itself.

User

ReLU layer: backward

Now implement the backward pass for the ReLU activation function. Note that the ReLU activation is not differentiable at 0, but typically we don't worry about this and simply assign either 0 or 1.

ChatGPT

Here's the correct implementation for the ReLU backward pass:

```
def relu_backward(dout, cache):  
    """  
    Computes the backward pass for a layer of rectified linear units (ReLU).  
    """  
    x = cache  
    dx = dout * (x > 0) # gradient is 1 where x > 0, else 0  
    return dx
```

Notes:

- The derivative of ReLU is 1 for positive inputs and 0 for non-positive inputs.
- Multiplying `dout` by $(x > 0)$ applies the chain rule.

User

Two-layer network

Open the file `deeplearning/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. The architecture should be affine - relu - affine - softmax.

ChatGPT

Here's a complete guide to implementing the `TwoLayerNet` class.

[Image of two-layer neural network architecture diagram]

1. Initialization

```
def __init__(self, input_dim=3*32*32, hidden_dim=100, num_classes=10,
```

```

        weight_scale=1e-3, reg=0.0):
self.params = {}
self.reg = reg

# Initialize weights with Gaussian noise
self.params['W1'] = weight_scale * np.random.randn(input_dim, hidden_dim)
self.params['b1'] = np.zeros(hidden_dim)
self.params['W2'] = weight_scale * np.random.randn(hidden_dim, num_classes)
self.params['b2'] = np.zeros(num_classes)

```

2. Forward pass (loss function, test-time)

```

def loss(self, X, y=None):
    # Unpack parameters
    W1, b1 = self.params['W1'], self.params['b1']
    W2, b2 = self.params['W2'], self.params['b2']

    # Forward pass: affine - ReLU - affine
    a1, cache1 = affine_forward(X, W1, b1)
    h1, relu_cache = relu_forward(a1)
    scores, cache2 = affine_forward(h1, W2, b2)

    if y is None:
        return scores

```

3. Backward pass (training-time)

```

    loss, grads = 0, {}

    # Compute softmax loss
    shifted_logits = scores - np.max(scores, axis=1, keepdims=True)
    Z = np.sum(np.exp(shifted_logits), axis=1, keepdims=True)
    log_probs = shifted_logits - np.log(Z)
    probs = np.exp(log_probs)
    N = X.shape[0]
    loss = -np.sum(log_probs[np.arange(N), y]) / N

    # Add L2 regularization
    loss += 0.5 * self.reg * (np.sum(W1*W1) + np.sum(W2*W2))

    # Backward pass
    dscores = probs.copy()
    dscores[np.arange(N), y] -= 1
    dscores /= N

    # Gradient w.r.t second layer
    dh1, dW2, db2 = affine_backward(dscores, cache2)
    dW2 += self.reg * W2 # regularization
    grads['W2'] = dW2
    grads['b2'] = db2

    # Gradient w.r.t first layer
    da1 = relu_backward(dh1, relu_cache)
    dx, dW1, db1 = affine_backward(da1, cache1)
    dW1 += self.reg * W1 # regularization
    grads['W1'] = dW1
    grads['b1'] = db1

    return loss, grads

```

User

Solver

Following a modular design, for this assignment we have split the logic for training models into a separate class. Open `deeplearning/solver.py`... Use a Solver instance to train a TwoLayerNet that achieves at least 50% accuracy on the validation set.

```
model = TwoLayerNet()
solver = None

# TODO: Use a Solver instance to train a TwoLayerNet that achieves at least
# 50% accuracy on the validation set.
```

ChatGPT

To solve this, we need to instantiate the TwoLayerNet, wrap it in the Solver class, and define the hyperparameters (learning rate, hidden dimension, regularization, etc.) that allow the model to converge.

Here is the implementation:

```
# Create the model
# Using a slightly larger hidden dimension can help with capacity
model = TwoLayerNet(hidden_dim=100, reg=0.01)

# Create the solver
solver = Solver(model, data,
                update_rule='sgd',
                optim_config={
                    'learning_rate': 1e-3,
                },
                lr_decay=0.95,
                num_epochs=10,
                batch_size=100,
                print_every=100)

# Train the model
solver.train()
```

Explanation of choices:

- `hidden_dim=100`: Provides enough capacity to learn features from CIFAR-10.
- `learning_rate=1e-3`: A standard starting point for SGD.
- `num_epochs=10`: Sufficient time for the network to reach >50% accuracy without taking too long.